# A Performance Analysis of Faster-R-CNN and YOLOv8 in Object Detection Using the 2017 COCO Dataset

**Research question:** How does Faster R-CNN compare with YOLOv8 in terms of speed and accuracy?

**Subject:** Computer Science

**Session:** May 2025

**Word count:** 3921

**Contents**

## 1. Introduction

Computer vision is a field that enables computers to interpret and extract information from images and videos. A key technique in this field is object detection, which allows computers to identify and locate objects within visual data. This technology plays a crucial role in applications like self-driving cars, security surveillance, facial recognition, and medical image analysis, where systems need to respond accurately to visual inputs. The effectiveness of object detection is typically measured by two primary metrics: accuracy (covering both classification and localization) and speed (Zou et al. 1).

Two prominent object detection models are Faster R-CNN and YOLOv8, each known for different strengths. Faster R-CNN, a two-shot detector, generates region proposals before classifying and refining them, resulting in high accuracy but slower processing. In contrast, YOLOv8, a one-shot detector, streamlines this process by simultaneously localizing and classifying objects, achieving faster results (Parab et al. 509). Recent advancements significantly improved the performance of models within the R-CNN and YOLO families. Notably, Faster R-CNN has seen remarkable improvements in speed and efficiency (Ren et al. 2), while the latest version of YOLO at the time of selecting this topic, YOLOv8, has achieved impressive gains in accuracy (Yaseen). This raises the crucial question: how do these two models compare in terms of overall performance?

To explore this, models trained on the same dataset will be evaluated on an appropriate test set to provide a fair comparison. The 2017 COCO dataset will be used as it is a benchmark for object detection due to its comprehensive diversity, realistic scenarios, and challenging complexities, providing a standardized and widely recognized platform for evaluating and comparing model performance across the computer vision community (Mupparaju et al. 11). This is an updated version of the original dataset that includes additional images and revised

annotations. For a detailed overview of the dataset's creation, refer to the original paper by Lin et al. As mentioned, understanding which model is superior depends on balancing accuracy with speed. Therefore, my research question is “How does Faster R-CNN compare with YOLOv8 in terms of speed and accuracy?”

The essay will outline the metrics used, explain the unique features of each model, describe the methodology, analyse the results, and evaluate the overall experiment. Although initial findings suggest that YOLOv8 outperforms Faster R-CNN in speed and overall accuracy, further analysis of other performance metrics will provide valuable insights into which model is best suited for specific applications.

## 2. Metrics

In object detection, speed is typically measured by inference time or frames per second (FPS), simply the inverse of time taken for a single inference. On the other hand, measuring accuracy is more complex, involving a variety of nuanced metrics.

According to Padilla et al., to assess a model's performance, its predictions are compared to ground truths, which are annotations indicating the actual content of the image. For a prediction to be deemed accurate, the model must correctly identify the object's location using a bounding box that encloses the object and also make the correct classification. Additionally, each detection is assigned a confidence score, reflecting the model's level of certainty in its prediction (7).

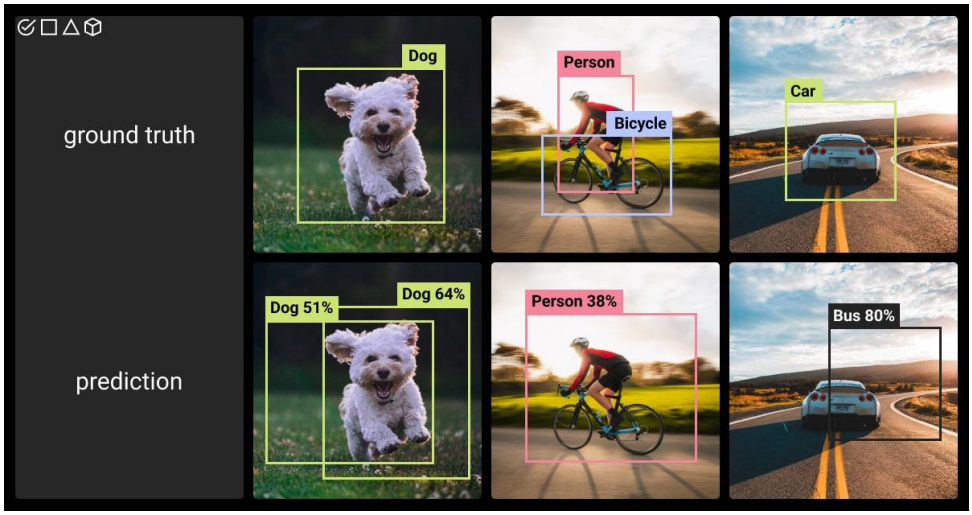


*Figure 1. Ground truth vs. Prediction (Knaizieva)*

Figure 1 shows a visualization of the ground truth compared to a model's prediction. The following metrics explain how the model is evaluated to determine its accuracy, with each metric ranging from 0 to 1.

## 2.1 Intersection over Union

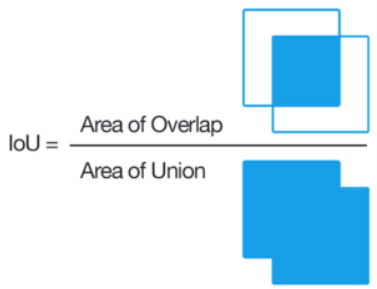


*Figure 2. Intersection over union (Rosebrock)*

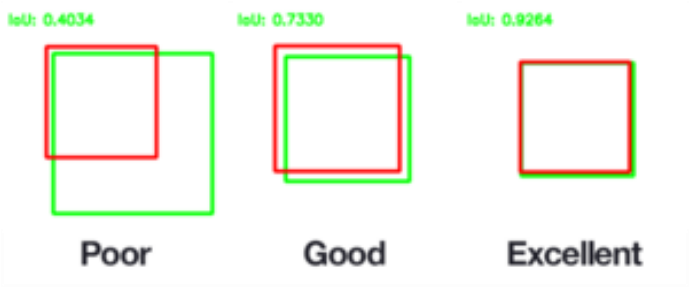


*Figure 3. An example of computing Intersection over Unions for various bounding boxes (Rosebrock)*

Intersection over Union (IoU) is a metric used to evaluate the accuracy of object detection models by measuring the overlap between the predicted bounding box and the ground truth bounding box. It is calculated as the ratio of the overlapping area between the predicted and true bounding boxes to the total area covered by both boxes. Setting an IoU threshold adjusts how strictly other metrics judge detections as correct or incorrect (Padilla et al. 8) This calculation is visualized in Figure 2. Figure 3 demonstrates how varying IoU values may be penalized.

In real life, it is challenging for a model to predict the exact bounding box of the ground truth accurately. Therefore, this way of calculation ensures that the model is rewarded for significant overlap while also being penalized if the predicted bounding box is significantly larger than the ground truth (“Intersection over Union (IoU) | CloudFactory Computer Vision Wiki”).

## 2.2 True Positives, False Positives and False Negatives

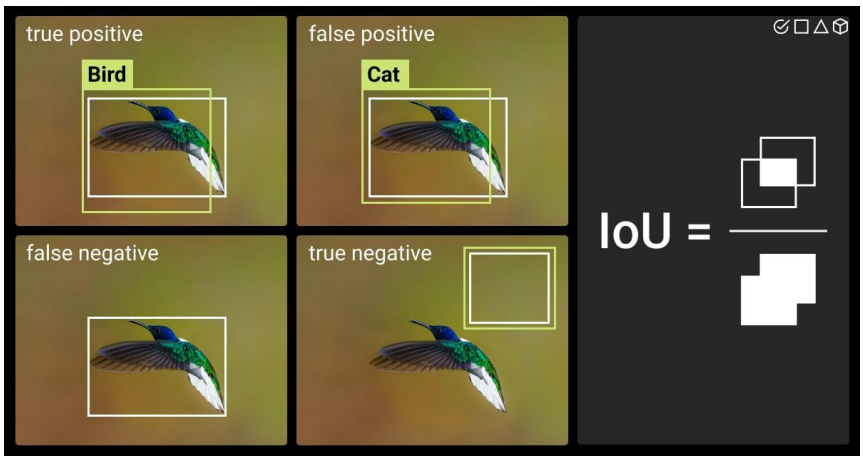


*Figure 4. True Positives (TP), False Negatives (FN), False Positives (FP), and True Negatives (TN) (Knaizieva)*

- A true positive (TP) occurs when the object detector correctly identifies an object and the predicted bounding box meets the IoU threshold set by the researcher.
- A false positive (FP) occurs when the object detector incorrectly identifies an object that is not present or when the IoU is below the threshold.
- A false negative (FN) occurs when the object detector fails to identify an object that is actually present in the image.

(Padilla et al. 9)

Figure 4 provides a visualization of the outlined metrics, where true negatives in the image can be disregarded as they are irrelevant in object detection.

## 2.3 Precision, Recall, and F1 Score

Precision indicates how many of the predicted positive instances are correct. Suppose you picked 10 fruits from the basket based on your belief that they are apples. If 8 out of these 10

fruits are actually apples, your precision is 0.8. Precision is like ensuring that what you pick is truly what you want and is calculated by:

$$Precision = \frac{TP}{TP + FP}$$

Now, consider there were 12 apples in the basket in total. You found 8 apples out of these 12. Therefore, your recall is about 0.67. Recall measures how well you've managed to gather all that is available and is calculated by:

$$Recall = \frac{TP}{TP + FN}$$

The F1 Score is a metric that evaluates how effectively a model balances precision and recall and is calculated by:

$$F1\ Score = \frac{2}{\frac{1}{Precision} + \frac{1}{Recall}} = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Rather than averaging these, which could be misleading, the F1 Score employs the harmonic mean. This approach adjusts for imbalances by focusing on the reciprocals of the metrics, which helps normalize their different bases before combining them. By doing so, it prevents either precision or recall from being disproportionately emphasized, particularly when one is significantly higher or lower than the other (GeeksforGeeks, “F1 Score in Machine Learning”).

## 2.4 Average Precision

The confidence score was mentioned, but its importance was not discussed. This score helps filter out less reliable predictions, reducing false positives by setting a threshold that only allows further processing of high-confidence detections. This feature is particularly important in applications with strict legal requirements or where the consequences of incorrect detections are severe, ensuring that only the most reliable detections influence decision-making (Wenkel et al. 1).

Metrics such as precision, recall, and F1 score are dependent on the confidence score threshold. As you modify the confidence threshold of the model, the values for precision and recall change. Plotting precision and recall at different confidence thresholds produces a precision-recall curve, where the Area Under the Curve (AUC) is interpreted as the Average Precision (AP). Lowering the confidence threshold typically increases recall by allowing more predictions, but this can also decrease precision as more FPs may occur ("Average Precision - Testing with Kolena").

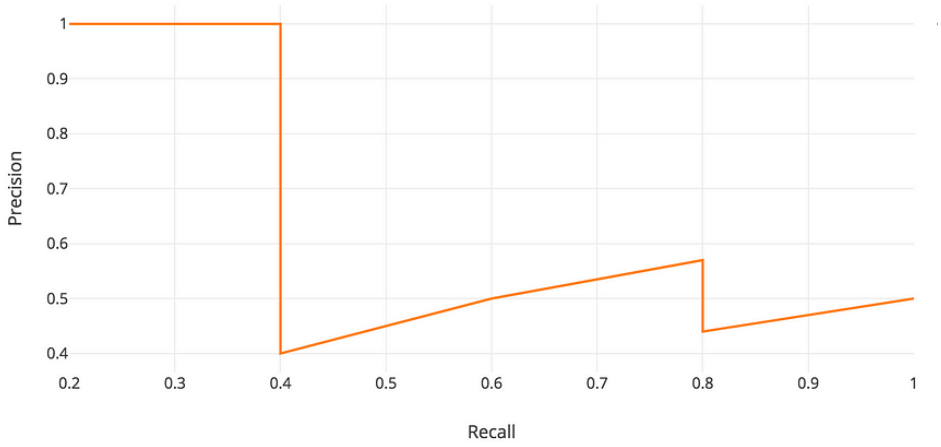


*Figure 5. Zigzag pattern of the precision-recall curve (Hui)*

A good object detection model should effectively identify all relevant objects while minimizing the identification of irrelevant ones and avoiding excessive redundant predictions. As the confidence threshold changes, the model should aim to maintain both high recall and high precision. Therefore, a higher AUC indicates better overall model performance. However, in practice, the precision-recall curve often exhibits a zigzag pattern as shown in Figure 5, due to fluctuations in precision as precision drops with FPs and rises with TPs as recall increases (Padilla et al. 9-11).


*Figure 6. Smoothening the pattern (Hui)*

To smooth this zigzag pattern, precision values are adjusted by taking the highest precision value to the right, resulting in a smoother curve that more accurately reflects the model's performance (Hui).

The AP for each class is calculated individually, and the mean of these AP values across all classes results in the Mean Average Precision (mAP). The mAP serves as a comprehensive evaluation metric that reflects a model’s overall performance across all classes (Padilla et al. 11).

## 3. Background

The following concepts will be simplified to highlight the key differences between the models without straying from the main research question. References for further learning will be provided for those interested in more detailed information.

### 3.1 Training

Object detection models must be trained before they can accurately predict objects in images. This training involves supplying the models with a large dataset of labelled examples known as ground truths. Initially, these models will perform poorly. However, as they process the data and make prediction errors, their internal parameters are adjusted. This adjustment is guided by a loss function, which quantifies the difference between the model's predictions and the actual target values in the dataset. Through this iterative process, the models learn to recognize and interpret various features of the data more accurately (“How Object Detectors Learn”). Inference, however, is done with a model which has already been trained to generate its predictions.

Unlike parameters that are set during training, most modern machine learning algorithms have parameters that need to be fixed before running them. Such parameters are referred to as hyperparameters. In many cases, the performance of an algorithm on a given learning task depends on its hyperparameter settings. To obtain the best performance, machine learning practitioners can tune the hyperparameters (Weerts et al. 1). Unfortunately, the link between the performance of machine learning algorithms and their hyperparameters is not always clear. In practice, it's essential to repeatedly tweak these hyperparameters and train various models using different value combinations. Afterward, the performance of each model is compared to identify the most effective one (Wu et al.). Hyperparameters vary across different models, and it is possible that one may be better tuned than the other.

### 3.2 Feature Extraction

Most object detection models consist of similar components: an input (such as an image), a backbone, a neck, and a head (Bochkovskiy et al. 2).

A critical stage in object detection is extracting relevant features from images, enabling the model to learn patterns and make accurate predictions. Backbone networks, typically pre-trained convolutional neural networks (CNNs), are essential in this feature extraction process. The performance of models like YOLOv8 and Faster R-CNN heavily depends on the quality of these feature extractors (Bouraya and Belangour 1379).

As the image passes through each layer of the CNN, filters are applied, each designed to detect different features, ranging from simple edges in the initial layers to more complex textures and patterns in the deeper layers. This hierarchical process is crucial for extracting detailed feature maps from the input image. The initial layers capture basic elements like edges, subsequent layers might identify textures or patterns formed by these edges, and deeper layers recognize more complex objects (Rey). For a deeper understanding, refer to Ghosh et al., while Rey provides an overview.

The neck module refines and fuses the multi-scale features extracted by the backbone, further transforming and enhancing these features. The head then utilizes these refined features to make the final predictions (Shroff). A feature pyramid network (FPN) neck is employed by Faster R-CNN and YOLOv8 and helps the model recognize objects by passing deep, semantic information from the upper layers down to the lower layers, enhancing object understanding. However, this top-down approach can lose some precise location details. To fix this, YOLOv8 adds a Path Aggregation Network (PAN) on top of the FPN. PAN introduces a bottom-up path, where information flows from lower to higher layers, adding back important localization details that may have been lost (Wang et al. 3).

The comparative analysis will acknowledge the roles of training, hyperparameters, and backbone networks in shaping the performance of the models but the comparison centers on each model's detection strategies during inference. This examination highlights how each model processes feature maps to classify and locate objects to determine which model best meets specific operational needs.

### 3.3 Faster R-CNN

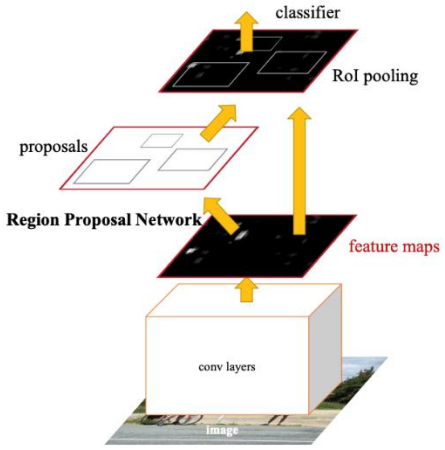


*Figure 7. Faster R-CNN architecture (Ren et al.)*

Faster R-CNN's two stages consist of the Region Proposal Network (RPN) and the Fast R-CNN detector, which serves as the head. The RPN and Fast R-CNN are integrated into a single network architecture. This means that both components utilize the same convolutional layers to process the input image. By sharing these layers, the network can leverage the same feature maps for both region proposal generation and object detection, reducing computational redundancy. Further reading can be found in Ren et al., which covers this area extensively (Ren et al.).

### 3.3.1 Region Proposal Network

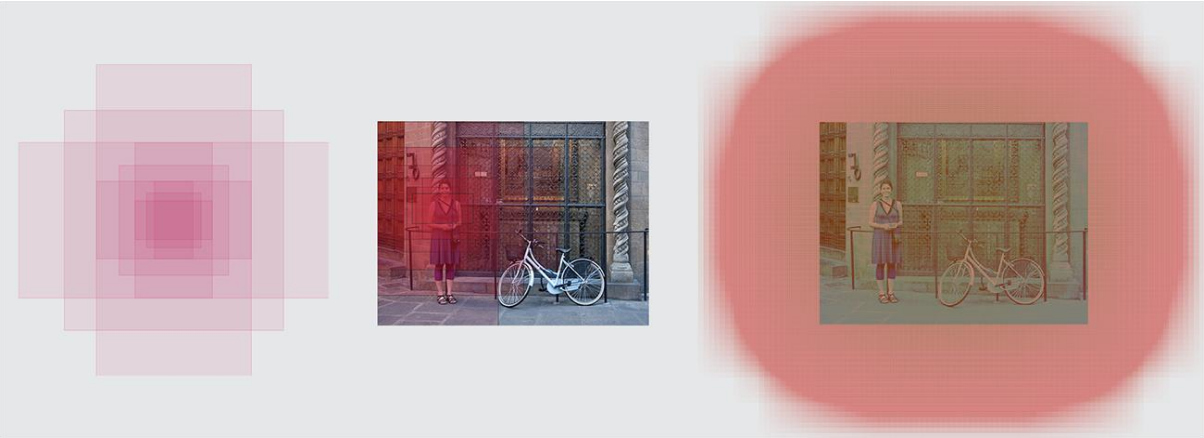

*Figure 8. Left: Anchors, Center: Anchor for a single point, Right: All anchors (Rey)*

Anchors are predefined bounding boxes that serve as references for predicting the locations of objects within an image. These boxes are designed to cover various scales and aspect ratios. These anchor boxes are tiled across the entire feature map. Overlapping anchors ensure that even objects that are not perfectly aligned with the grid of the feature map can be detected. Figure 8 visualizes how different scales and aspect ratios of anchors are applied across the feature map, as represented on the input image. Anchors allow the model to handle objects of different dimensions and at different positions within the image.

The RPN in Faster R-CNN uses anchors as reference points to generate region proposals, indicating potential object locations. Each anchor box is evaluated and assigned objectness scores, which are probability values indicating whether it likely contains an object or not. Additionally, the bounding box regression predicts adjustments to refine its position and size (Ren et al. 3).

### 3.3.2 Non-Maximum Suppression

Many of the proposals generated by the RPN overlap, especially around the same object. Non-Maximum Suppression (NMS) addresses this by sorting proposals based on their objectness scores and eliminating those with significant overlap. This step ensures that only the most confident and non-overlapping proposals are kept, reducing redundancy, which enhances both speed and accuracy (Ren et al. 7).

### 3.3.3 Fast R-CNN detector

#### 3.3.3.1 Region of Interest Pooling

The Region of Interest (RoI) pooling layer converts variable-sized proposals into fixed-size feature maps, enabling uniform processing. RoI pooling reshapes all proposals to a consistent size, which is essential for further processing by the subsequent layers (GeeksforGeeks, “Faster R-CNN | ML”).

#### 3.3.3.2 Classification and Bounding Box Regression

The classification layer predicts class probabilities for each proposal, including a background class. The regression layer refines the bounding box coordinates, making them more accurate by predicting further adjustments needed for the centre coordinates and dimensions of the bounding box (GeeksforGeeks, “Faster R-CNN | ML”).

### 3.3.4 Post-processing

Like the RPN, this process results in several objects identified with assigned classes that require additional refinement. For adjusting the bounding boxes, it's necessary to determine the class with the highest probability for each proposal. Proposals where the background class is most likely are disregarded. Once the definitive objects are identified and those deemed

background are excluded, class-specific NMS is applied. This involves grouping the objects by their class, ordering them by probability, and executing NMS on each group independently (Rey).

## 3.4 YOLOv8

There is no official paper on YOLOv8 available yet. However, readers interested in learning more can refer to Mupparaju et al.

### 3.4.1 Mosaic and Mix-up Augmentation

YOLOv8 uses mosaic and mix-up data augmentation during training, where four or more images are blended into one training example. This technique exposes the model to varied object configurations, scales, and backgrounds in a single image, improving its ability to recognize objects in complex scenes. Such augmentation boosts robustness and performance and improves generalization on new unseen data (Yaseen).

### 3.4.2 Anchor-free detection

YOLOv8 forgoes traditional anchors to predict bounding box coordinates; instead of calculating offsets from predefined anchor boxes, it predicts the object center directly, improving the model's adaptability to objects with varying aspect ratios and scales (Yaseen). By removing the need for multiple predefined anchors at each feature map location, YOLOv8 reduces computational complexity and speeds up NMS process (Mupparaju et al. 6)

### 3.4.3 Head Module

YOLOv8 uses a decoupled head architecture, separating the tasks of objectness score, classification, and bounding box regression. This separation often leads to more accurate predictions and efficient learning, as each head can optimize independently without interference from the other (Mupparaju et al. 6).

### 3.4.4 Post-Processing

After making predictions, YOLOv8 uses post-processing techniques like Faster R-CNN. NMS removes duplicate or overlapping bounding boxes, ensuring that only the most accurate detections remain (Boesch).

## 4. Methodology

### 4.1 Experimental Setup

The experiment will use Google Colab, a cloud-based Jupyter notebook environment commonly used for machine learning projects. Google Colab is integrated with Google Drive, allowing for easy data loading and storage. It comes with several essential libraries pre-installed (Sandika S. Sukhdeve and Dr. Shitalkumar Sukhdeve 1) and is compatible with Python, which is widely used in machine learning due to its data-handling capabilities (Balaraman 1-2). In addition, it provides access to powerful graphical processing units (GPUs), which can execute computationally exhaustive code. I have selected the Nvidia T4 GPU as resources on colab are limited and scarce (Google) and this GPU is sufficient for inference (“NVIDIA T4 Tensor Core GPUs for Accelerating Inference”)

### 4.2 Models and Dataset

The study uses PyTorch's Faster R-CNN with a ResNet-50 backbone and FPN (TorchVision maintainers and contributors) as outlined in the official documentation (Pytorch) and YOLOv8 developed by Ultralytics (Jocher et al.). Both models are pre-trained on the 2017 COCO dataset. YOLOv8 employs a modified version of CSP-Darknet-53 as its backbone (Mupparaju et al.). There are different versions of the YOLOv8 model, YOLOv8n being the least complex, making it faster but less accurate than its counterparts (Yaseen). It has been selected in this study to save computational resources.

To test the models, I will use COCO val2017 as it is a popular benchmark dataset (Mupparaju et al. 11). It is the validation dataset that is used for tuning hyperparameters after training. Ideally, a different dataset should be used for testing to provide unbiased results (GeeksforGeeks, "Why Do We Use Both Validation Set and Test Set?"). Since the testing dataset's annotations aren't available for public use, the validation dataset has become a benchmark dataset. Another limitation of this dataset is that some classes are overrepresented (Solawetz). As a result, the model may overfit those classes, meaning that it performs well on the dominant class but poorly on the others (Shah). This makes it difficult to accurately assess a model's performance across all classes, which is crucial when working with a diverse dataset like COCO.

### 4.3 Model Inference and Evaluation

For each image in the dataset, inference was run separately for both models to generate predictions. A confidence threshold of 0.5 was set to filter out predictions with lower likelihoods of accuracy. At the same time, the time for inference is measured per image. To accurately assess the performance of our model, we need to identify TPs, FPs, and FNs (see page 7). There are several approaches to match detections with ground truths. One method is

to iterate over the predictions in an image and match each one with the ground truth box that has the highest IoU (see page 5), ensuring that the class prediction is correct. It is crucial to maintain a one-to-one mapping, as multiple predictions cannot be matched to a single ground truth and vice versa (Lalovic). I implemented this by first sorting predictions in descending order of confidence to prioritize those with a higher probability of being correct. Additionally, I set an IoU threshold of 0.5 to add an extra constraint for a prediction to be deemed a TP. The exact code is available in Appendix A for further reference.

With these categorizations in place, we can directly calculate precision, recall, and the F1 score. The average IoU is determined by taking the mean of all IoUs from the matched predictions, while the average time is calculated from the detection times. The skicit-learn library was used to calculate the APs of each (Pedregosa et al.). To reduce computational intensity, I applied a confidence threshold of 0.001, similar to the approach used by Ultralytics (Ultralytics, "Configuration"), instead of considering all predictions.

## 5. Results

The results of this study show that YOLOv8n and Faster R-CNN each have distinct strengths and weaknesses, making them suitable for different applications depending on the specific requirements for speed and accuracy.

| Model | TP | FP | FN | Precision | Recall | F1 Score | Average IoU | mAP | Average time (seconds) | FPS |
|---|---|---|---|---|---|---|---|---|---|---|
| Faster R-CNN | 26577 | 18456 | 10204 | 0.590 | 0.723 | 0.650 | 0.888 | 0.755 | 0.0853 | 11.7 |
| Yolov8n | 13675 | 1552 | 23106 | 0.898 | 0.372 | 0.526 | 0.928 | 0.936 | 0.0351 | 28.5 |

*Table 1. Results rounded to three significant figures*

YOLOv8n significantly outpaces Faster R-CNN in speed, operating nearly three times faster, which makes it ideal for real-time applications like autonomous vehicles and surveillance systems. This speed advantage is primarily due to its anchor-free detection system, which reduces redundancy and accelerates NMS, as well as its single-shot detection approach that predicts object classifications and locations simultaneously in one pass through the network. Moreover, YOLOv8n exhibits a slightly better average IoU compared to Faster R-CNN, likely because its anchor-free detection allows greater flexibility in adapting to various object shapes and sizes, resulting in more precise bounding box placements.

Faster R-CNN is designed to generate a multitude of proposals to maximize the chances of detecting potential objects, significantly enhancing its recall. This model ends up making more predictions (sum of TPs and FPs), than YOLOv8n, resulting in a higher number of true positives and fewer false negatives. Such capabilities are especially valuable in scenarios where the cost of missing an object is high, such as in medical imaging for tumour detection or in autonomous driving environments where ensuring the detection of every pedestrian and vehicle is critical for safety. However, the downside of this strategy is an increase in false positives, as the model processes and refines each proposal regardless of its initial likelihood of containing an actual object.

On the other hand, YOLOv8n utilizes an anchor-free detection system that inherently reduces redundancy by avoiding overlapping detection boxes. This approach leads to fewer overall predictions, significantly reducing FPs—almost by a factor of ten compared to Faster R-CNN. This level of precision is critical in applications like autonomous driving, where incorrectly identifying a harmless object as a threat could trigger unnecessary and potentially dangerous maneuvers, such as abrupt braking. Nonetheless, the lower number of predictions from YOLOv8n also leads to more FNs, thereby decreasing its recall relative to Faster R-CNN.

However, Faster R-CNN achieves a higher F1 score, indicating a better balance between precision and recall at a high confidence threshold of 0.5. This metric is crucial in applications where both detecting as many objects as possible and ensuring detections are correct are equally important. An example might include security systems, where overlooking a genuine threat or responding to a false alarm both carry significant consequences.

YOLOv8n's higher mAP reflects that it can adapt more flexibly to different threshold settings without a significant drop in performance. This is particularly valuable in dynamic environments where conditions may change, requiring adjustments to how detections are prioritized. Given that mAP is the standard measure of overall accuracy, this suggests that YOLOv8n outperforms Faster R-CNN in terms of accuracy.

Its anchor-free detection system, combined with advanced data augmentations like MixUp and Mosaic, and the enhanced PAN neck, likely contribute to its superior mAP. These features improve feature aggregation and robustness, enhancing detection performance across various classes and object sizes and making YOLOv8n more versatile and effective in diverse detection scenarios.

## 6. Conclusion

To revisit the research question, "How does Faster R-CNN compare with YOLOv8 in terms of speed and accuracy?" it has been determined that YOLOv8 outperforms Faster R-CNN in both speed and mAP under the conditions defined for this study. When looking at inference time and mAP at an IoU threshold of 0.5, YOLOv8 clearly excels. Both models show impressive localization accuracy, evidenced by their similar and high mean IoUs. A closer look at other metrics reveals that at a high confidence threshold of 0.5, Faster R-CNN surpasses YOLOv8 in recall and F1 score, indicating a better balance of precision and recall at this threshold, whereas YOLOv8 achieves higher precision. These results can be traced back to differences in their architectures, which can help guide the choice of a model based on specific use-case requirements. The analysis successfully answers the research question, although it only considers one variant of YOLOv8, YOLOv8n. The performance of a more complex variant of YOLOv8 and whether it would maintain its speed advantage remains untested.

Nevertheless, there are limitations in the study that could affect the overall conclusions. Variations in hyperparameter tuning, training conditions, and backbone architectures could potentially skew the comparison in favor of one model. For example, we cannot conclude whether a different backbone might have changed the outcome. The study also does not examine how the models would perform under stricter IoU and confidence thresholds. Additionally, the study did not assess model performance under stricter IoU and confidence thresholds. The choice of dataset is another potential weakness; class imbalances in the COCO dataset could skew the evaluation, as models may overfit overrepresented classes while underperforming on underrepresented ones. Using the validation set instead of a test set

may also introduce bias, as it does not accurately reflect a model's performance on truly unseen data.

## 7. Works Cited


"Average Precision - Testing with Kolena." *Kolena.com*, 24 Sept. 2024, docs.kolena.com/metrics/average-precision/#:~:text=Average%20precision%20(AP)%20summarizes%20a. Accessed 25 Sept. 2024.

Balaraman, Sundarambal. *Comparison of Classification Models for Breast Cancer Identification Using Google Colab*. 20 May 2020, www.preprints.org/manuscript/202005.0328/v1, https://doi.org/10.20944/preprints202005.0328.v1. Accessed 26 Sept. 2024.

---. *Comparison of Classification Models for Breast Cancer Identification Using Google Colab*. 20 May 2020, www.preprints.org/manuscript/202005.0328/v1, https://doi.org/10.20944/preprints202005.0328.v1. Accessed 26 Sept. 2024.

Bochkovskiy, Alexey, et al. "YOLOv4: Optimal Speed and Accuracy of Object Detection." *ArXiv*, 22 Apr. 2020, arxiv.org/abs/2004.10934, https://doi.org/10.48550/arXiv.2004.10934%20Focus%20to%20learn%20more. Accessed 25 Sept. 2024.

Boesch, Gaudenz. "A Guide to YOLOv8 in 2024." *Viso.ai*, 18 Dec. 2023, viso.ai/deep-learning/yolov8-guide/. Accessed 19 Aug. 2024.

Bouraya, Sara, and Abdessamad Belangour. "Object Detectors' Convolutional Neural Networks Backbones : A Review and a Comparative Study." *International Journal of Emerging Trends in Engineering Research*, vol. 9, no. 11, 7 Nov. 2021, pp. 1379–1386, www.researchgate.net/publication/379567858_Object_Detectors'_Convolutional_Neural_Networks_backbones_a_review_and_a_comparative_study, https://doi.org/10.30534/ijeter/2021/039112021. Accessed 26 Sept. 2024.

GeeksforGeeks. “F1 Score in Machine Learning.” *GeeksforGeeks*, www.geeksforgeeks.org/f1-score-in-machine-learning/. Accessed 25 Sept. 2024.

---. “Faster R-CNN | ML.” *GeeksforGeeks*, 27 Feb. 2020, www.geeksforgeeks.org/faster-r-cnn-ml/. Accessed 24 June 2024.

Jocher, Glenn et al. *Ultralytics YOLO*. Version 8.0.0, Ultralytics, 10 Jan. 2023, https://github.com/ultralytics/ultralytics. Accessed 26 Sept. 2024

Pedregosa, Fabian, et al. "Scikit-learn: Machine Learning in Python." *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825-2830.

TorchVision maintainers and contributors. *TorchVision: PyTorch's Computer Vision Library*. GitHub, 2016, https://github.com/pytorch/vision.

---. “Why Do We Use Both Validation Set and Test Set?” *GeeksforGeeks*, 13 Feb. 2024, www.geeksforgeeks.org/why-do-we-use-both-validation-set-and-test-set/. Accessed 27 Sept. 2024.

Ghosh, Anirudha, et al. “Fundamental Concepts of Convolutional Neural Network.” *Intelligent Systems Reference Library*, Jan. 2020, pp. 519–567, www.researchgate.net/publication/337401161_Fundamental_Concepts_of_Convolutional_Neural_Network, https://doi.org/10.1007/978-3-030-32644-9_36. Accessed 26 Sept. 2024.

Google. “Colaboratory – Google.” *Research.google.com*, Google, research.google.com/colaboratory/faq.html. Accessed 26 Sept. 2024.

“How Object Detectors Learn.” *Ambolt*, ambolt.io/en/how-object-detectors-learn/. Accessed 24 Sept. 2024.

Hui, Jonathan. “MAP (Mean Average Precision) for Object Detection.” *Medium*, 3 Apr. 2019, jonathan-hui.medium.com/map-mean-average-precision-for-object-detection-45c121a31173. Accessed 27 Sept. 2024.

"Intersection over Union (IoU) | CloudFactory Computer Vision Wiki." *CloudFactory Computer Vision Wiki*, wiki.cloudfactory.com/docs/mp-wiki/metrics/iou-intersection-over-union. Accessed 26 Sept. 2024.

Knaizieva, Yuliia. *LabelYourData*, 13 Aug. 2023, labelyourdata.com/articles/object-detection-metrics. Accessed 27 Sept. 2024.

Lalovic, Marko. "How to Define Precision When We Have Multiple Predictions for Each Ground Truth Instance?" *Cross Validated (Stack Exchange)*, 17 Feb. 2024, stats.stackexchange.com/questions/639424/how-to-define-precision-when-we-have-multiple-predictions-for-each-ground-truth#:~:text=Greedy%20matching%20is%20commonly%20employed. Accessed 26 Sept. 2024.

Lin, Tsung-Yi, et al. "Microsoft COCO: Common Objects in Context." *ArXiv.org*, 2014, arxiv.org/abs/1405.0312. Accessed 27 Sept. 2024.

Mupparaju, Sohan, et al. "A Review on YOLOv8 and Its Advancements." *Algorithms for Intelligent Systems*, Jan. 2024, pp. 529–545, www.researchgate.net/publication/377216968_A_Review_on_YOLOv8_and_Its_Advancements, https://doi.org/10.1007/978-981-99-7962-2_39. Accessed 26 Sept. 2024.

"NVIDIA T4 Tensor Core GPUs for Accelerating Inference." *NVIDIA*, www.nvidia.com/en-us/data-center/tesla-t4/. Accessed 27 Sept. 2024.

Padilla, Rafael, et al. "A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit." *Electronics*, vol. 10, no. 3, 25 Jan. 2021, p. 279, www.mdpi.com/2079-9292/10/3/279, https://doi.org/10.3390/electronics10030279. Accessed 26 Sept. 2024.

Parab, Chinmay U., et al. "Comparison of Single-Shot and Two-Shot Deep Neural Network Models for Whitefly Detection in IoT Web Application." *AgriEngineering*, vol. 4, no.

2, 10 June 2022, pp. 507–522, www.mdpi.com/2624-7402/4/2/34/pdf, https://doi.org/10.3390/agriengineering4020034. Accessed 21 Sept. 2024.

Pytorch. “Fasterrcnn_resnet50_fpn — Torchvision 0.12 Documentation.” *Pytorch.org*, pytorch.org/vision/0.12/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn.html. Accessed 25 Sept. 2024.

Ren, Shaoqing, et al. “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.” *ArXiv.org*, 4 June 2015, arxiv.org/abs/1506.01497. Accessed 26 Sept. 2024.

Rey, Javier. “Faster R-CNN: Down the Rabbit Hole of Modern Object Detection.” *Tryolabs*, 18 Jan. 2018, tryolabs.com/blog/2018/01/18/faster-r-cnn-down-the-rabbit-hole-of-modern-object-detection. Accessed 26 Sept. 2024.

Rosebrock, Adrian . *PyImageSearch*, 6 Nov. 2016, pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/. Accessed 25 Sept. 2024.

Shah, Deval. “COCO Dataset: All You Need to Know to Get Started.” *Www.v7labs.com*, 19 Jan. 2023, www.v7labs.com/blog/coco-dataset-guide. Accessed 26 Sept. 2024.

Shroff, Megha. “Object Detection with Deep Learning 2023: Beginner’s Friendly Key Terms Explanation.” *Medium*, 26 Mar. 2023, medium.com/@shroffmegha6695/object-detection-with-deep-learning-beginners-friendly-key-terms-explanation-d4fb594fea83. Accessed 26 Sept. 2024.

Solawetz, Jacob . “An Introduction to the COCO Dataset.” *Roboflow Blog*, 18 Oct. 2020, blog.roboflow.com/coco-dataset/. Accessed 26 Sept. 2024.

Sukhdeve , Dr. Shitalkumar, and Sandika S. Sukhdeve. “Google Colaboratory.” *Apress, Berkeley, CA*, 18 Nov. 2023, pp. 11–34, https://doi.org/10.1007/978-1-4842-9688-2_2.

Ultralytics. “Configuration.” *Ultralytics.com*, 12 Nov. 2023, docs.ultralytics.com/usage/cfg/#validation-settings. Accessed 26 Sept. 2024.

Wang, Xueqiu, et al. “BL-YOLOv8: An Improved Road Defect Detection Model Based on YOLOv8.” *Sensors*, vol. 23, no. 20, 10 Oct. 2023, www.mdpi.com/1424-8220/23/20/8361, https://doi.org/10.3390/s23208361. Accessed 26 Sept. 2024.

Weerts, Hilde J. P., et al. “Importance of Tuning Hyperparameters of Machine Learning Algorithms.” *ArXiv:2007.07588 [Cs, Stat]*, 15 July 2020, arxiv.org/abs/2007.07588, https://doi.org/10.48550/arXiv.2007.07588. Accessed 24 Sept. 2024.

Wenkel, Simon, et al. “Confidence Score: The Forgotten Dimension of Object Detection Performance Evaluation.” *Sensors*, vol. 21, no. 13, 25 June 2021, p. 4350, www.mdpi.com/1424-8220/21/13/4350, https://doi.org/10.3390/s21134350. Accessed 25 Sept. 2024.

Wu, Jia, et al. “Hyperparameter Optimization for Machine Learning Models Based on Bayesian Optimizationb.” *Journal of Electronic Science and Technology*, vol. 17, no. 1, Mar. 2019, pp. 26–40, www.sciencedirect.com/science/article/pii/S1674862X19300047, https://doi.org/10.11989/JEST.1674-862X.80904120. Accessed 26 Sept. 2024.

Yaseen, Muhammed. “What Is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector.” *Arxiv.org*, 24 Aug. 2024, arxiv.org/html/2408.15857v1/. Accessed 26 Sept. 2024.

Zou, Zhengxia, et al. “Object Detection in 20 Years: A Survey.” *ArXiv.org*, 13 May 2019, arxiv.org/abs/1905.05055. Accessed 21 Sept. 2024.

# 8. Appendices

## 8.1 Appendix A

https://github.com/yolov8fasterr-cnn/Appendix-A.git

Description: This GitHub repository contains the code used to evaluate and calculate the performance metrics of Faster R-CNN and YOLOv8n models. It covers how I derived the values for average time, average intersection over union, mean average precision, precision, recall, and F1 score.